Input Reduction Enhanced LLM-based Program Repair

Boyang Yang*†
School of Artificial Intelligence(School of Software), Yanshan University
China
yby@ieee.org

Luyao Ren*
School of Computer Science, Peking University
China
rly@pku.edu.cn

Xin Yin
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
China
xyin@zju.edu.cn

Jiadong Ren†
School of Artificial Intelligence(School of Software), Yanshan University
China
jdren@ysu.edu.cn

Haoye Tian‡
Department of Computer Science, Aalto University
Finland
tianhaoyemail@gmail.com

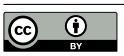
Shunfu Jin†
School of Artificial Intelligence(School of Software), Yanshan University
China
jsf@ysu.edu.cn

Abstract

Large Language Models (LLMs) have shown great potential in Automated Program Repair (APR). Test inputs, being crucial for reasoning the root cause of failures, are always included in the prompt for LLM-based APR. Unfortunately, LLMs struggle to retain key information in long prompts. When the test inputs are extensive in the prompt, this may trigger the "lost-in-the-middle" issue, compromising repair performance. To address this, we propose ReduceFix, an LLM-based APR approach with a built-in component that automatically reduces test inputs while retaining their failure-inducing behavior. ReduceFix prompts an LLM to generate a reducer that minimizes failure-inducing test inputs without human effort, and then feeds the reduced failure-inducing inputs to guide patch generation.

For targeted evaluation, we constructed LFTBench, the first long-input APR benchmark with 200 real bugs from 20 programming tasks, each paired with a failure-inducing input whose median size is 1 MB. On this benchmark, ReduceFix shrinks inputs by 89.1% on average and improves overall pass@10 by up to 53.8% relative to a prompt that includes the original test, and by 17.6% compared with omitting the test entirely. Adding the same reduction step to ChatRepair and CREF increases their fix rate by 21.3% and 2.6%, respectively, without other changes. Our gains hold against a ddmin-only reducing template baseline and transfer to repository-level OSS-Fuzz cases. Ablation studies further highlight the impact of input length and compressed failure information on repair success. These results underscore that automatically reducing failing inputs is a practical and powerful complement to LLM-based APR, significantly improving its scalability and effectiveness.

*Co-first authors who contributed equally to this work.
†Also affiliated with Hebei Key Laboratory of Computer Virtual Technology and System Integration.
‡Corresponding author.



ICSE '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2025-3/2026/04
https://doi.org/10.1145/3744916.3787760

CCS Concepts

• **Software and its engineering** → *Software defect analysis*; Software testing and debugging.

Keywords

Program Repair, Large Language Model, Input Reduction

ACM Reference Format:
Boyang Yang, Luyao Ren, Xin Yin, Jiadong Ren, Haoye Tian, and Shunfu Jin. 2026. Input Reduction Enhanced LLM-based Program Repair. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3744916.3787760

1 Introduction

APR aims to automatically generate bug-fixing patches for software defects, thereby reducing the manual effort required for debugging [1, 15, 39, 42]. Recently, LLMs have been applied to APR with promising results [19, 37]. Many recent APR systems enhance patch generation by including test inputs in the prompt, among which the test suite plays a particularly important role [14, 31, 43]. It provides a concrete example of how the program succeeds or fails, helping the LLM focus on the underlying issue and produce a correct fix. This strategy has shown strong results in recent systems such as ChatRepair [38]. Existing APR studies typically evaluate on benchmarks such as Defects4J [10] and HumanEval-Java [9], where test inputs are short and rarely exceed a few hundred characters. However, when the test input becomes too long, it is difficult to pinpoint the root cause of the error, known as the "lost-in-the-middle" phenomenon [18, 26], where information buried in a long prompt receives little attention and overall task performance drops [32, 43]. Therefore, an automatic test input reduction step becomes essential before repair.

Existing works on test input reduction rely heavily on human effort, which can be divided into two main categories.

Syntax-based approaches, such as HDD [22] and Perses [29], are built on the classical *ddmin* algorithm [48] but incorporate grammar or tree structure to ensure syntactic validity during reduction. However, adapting these APR tools to new tasks requires designing a new grammar and tuning heuristics for each input format, which

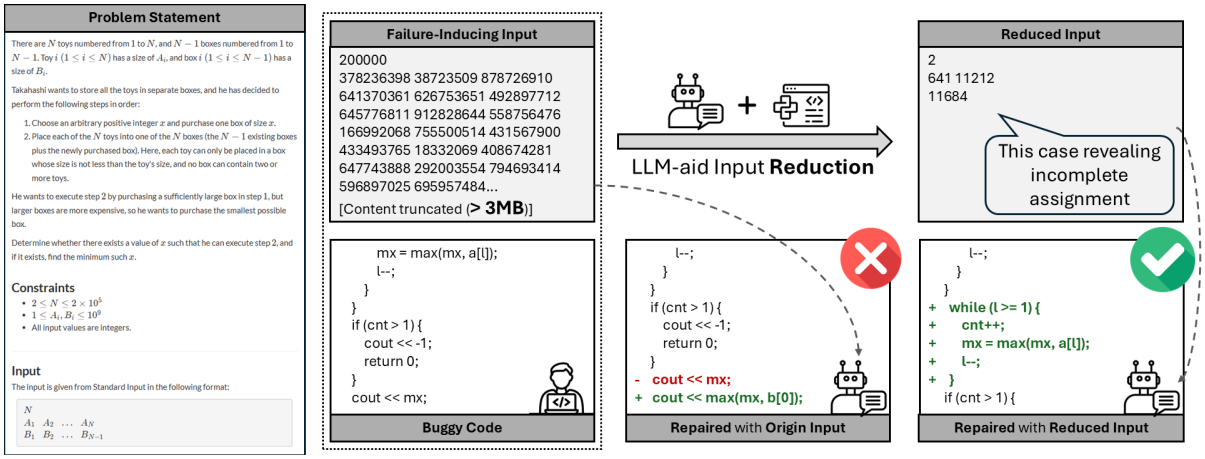


Figure 1: Motivating example (ABC376C): input reduction shrinks a >3MB failure-inducing test to three critical lines, enabling the LLM to generate the accurate patch.

is both time-consuming and error-prone. Other techniques, such as ddSMT [24] and J-Reduce [12], target specific domains, but still require domain knowledge and significant manual effort to implement and are not reusable across different formats. In LLM-based APR, the input format varies widely across tasks, often involving different formats ranging from plain text to structured JSON or domain-specific encoding [17]. As a result, relying on hand-crafted reducers is not scalable and makes automation difficult.

These observations uncover *two major limitations*:

① **Lack of length-aware handling for test input.** Although many APR systems embed the full test input into the prompt, few consider how input length affects patch quality. CREF [43] finds that on Bard, prompting with the failing test can hurt repair success, achieving 10% lower accuracy than the no-test baseline in some cases, primarily because long inputs overwhelm the LLM and trigger the "lost-in-the-middle" effect [18]. However, existing APR systems treat the test input as a fixed block of context and have not attempted to shorten or distill it before repair.

② **Limitation of manually crafted input reducers.** Prior approaches on input reduction often relies on handwritten syntax grammars or domain-specific rules, which require significant human effort and deep domain knowledge [22, 24, 29]. Even after laborious manual effort, each reducer remains tightly coupled to a specific task or file structure and cannot be generalized across formats. As LLM-based APR must handle a wide range of input types, including plain text, JSON, and custom encoding, manual reducers do not scale, and no existing method supports automatic reducer generation across diverse tasks.

To tackle the above limitations, we present ReduceFix, an LLM-based program repair framework that integrates automated input reduction into the repair loop. The framework prompts the LLM to customize a task-specific reducer, and then applies this reducer to produce a reduced failure-inducing input, finally leverages the reduced input to guide LLMs in generating the correct patch. ReduceFix thereby mitigates the "lost-in-the-middle" effect by reduction on each test case, allowing the repair model to concentrate on the actual failure-related parts instead of unrelated context. The overall process follows a three-stage pipeline: (i) reducer generation via a one-shot prompt with the problem description, (ii) time-bounded execution of the generated reducer to obtain the reduced input, and (iii) patch generation where the reduced input, buggy code, and problem description are jointly passed to the LLM.

To enable rigorous and leakage-free evaluation, we build LFTBench, the first APR benchmark that focuses on long test inputs, and we also provide a Python mirror, LFTBench-Py, with wrong-answer Python submissions. LFTBench contains 200 buggy codes from 20 AtCoder tasks released after the training cut-off dates of the evaluated LLMs. On LFTBench, ReduceFix with Qwen2.5-Plus generates syntactically correct reducers for all the 200 bugs, and 95.0% of those reducers successfully shrink the failure-inducing input by an average of 89.1%. ReduceFix with 4 selected LLMs increases their overall pass@10 by up to 53.8% compared with prompts that embed the whole test, and the advantage holds on LFTBench-Py. Beyond AtCoder, we validate on 12 OSS-Fuzz crash instances from five projects. Under Docker-grounded validation with Qwen2.5-Plus, reduced inputs raise the micro-average pass@10 to 41.7% from 25.0% over the no-test Baseline, and from 16.7% over the unreduced Origin Test. Replacing the reduced input for the original input in ChatRepair [38] raises its pass@10 by 21.3%, and integrating the same plug-in reducer into CREF increases its pass@10 by 2.6% relative, confirming that ReduceFix can be plugged into existing APR pipelines for an immediate accuracy boost.

The main contributions of our work are as follows:

- **Hands-free APR loop with integrating input reduction.** We design a repair framework ReduceFix that prompts an LLM to generate an input reducer to reduce the failure-inducing input and feeds the reduced input to the LLMs for patch generation.
- **Benchmark.** We release the first APR benchmark, LFTBench, which contains 200 bugs across 20 different real-world programming tasks, and a Python mirror LFTBench-Py for cross-language validation.
- **Comprehensive evaluations.** On LFTBench, ReduceFix produces reducers for all bugs and successfully reduces inputs

in 95.0% of cases with an average size reduction of 89.1%. Reduced tests raise *pass@10* by up to 53.8% across four LLMs. Comparisons to ddmin-only and pure LLM reducers show clear advantages in both success rate and cost. On LFTBench-Py, the reduced tests also outperform both the no-test and full-test prompts. On OSS-Fuzz, reduced inputs improve crash preservation and increase pass@10 from Origin Test's 16.7% to 41.7% under Docker-grounded validation. We further run ablations on prompt length and evidence composition, finding that shortening the failing input yields the largest gains, while appending only output diffs offers limited benefit.

- **Plug-in integration into existing APR systems.** We add ReduceFix as a drop-in reducer to ChatRepair and CREF without changing their logic. ChatRepair's pass@10 rises by 21.3% and CREF's pass@10 increases by 2.6% relative, showing that ReduceFix is immediately useful as a portable component.

2 Motivating Example

A single extensive test input can overwhelm an LLM with tokens and conceal the actual defect, which may be located deep within the prompt. This "lost-in-the-middle" effect lowers the attention paid to the critical lines and prevents the model from producing an accurate patch. For example, Problem C of AtCoder Beginner Contest 376 makes the issue concrete. The task receives two sorted sequences, A and B, and must print the maximum element of A that is not matched by any element of B, or -1 when more than one element remains unmatched. Figure 1 sketches the whole scenario.

The failed submission walks the two sequences from the back, increments `cnt`, and records `mx` whenever $a[l]$ is larger than $b[r]$. If $|A| > |B|$, the while-loop exits before the tail of A is checked, so some mismatches slip through.

The online judge provides a failure-inducing input that exceeds 3 MB. When Qwen2.5-Coder-7B-Instruct is prompted with the task statement, the buggy code, and this complete file, it adds only a superficial guard and still ignores the trailing elements, so the program continues to fail. The tokens that expose the oversight sit near the midpoint of the 3 MB prompt, far from either end, where the model focuses most of its capacity.

As shown in Figure 1, ReduceFix addresses this by inserting an automatic reduction stage before repair. It prompts Qwen2.5-Plus once to write a task-specific Python script that leverages the classical *ddmin* algorithm [48] and needs no extra manual effort. Running the LLM-generated reducer reduces the 3 MB test to a three-line counterexample that still forces the buggy and reference programs to diverge.

With this compact input, the same repair model introduces a second loop that scans the remaining part of A and updates `cnt` and `mx`. The patched program then passes every official test.

Long failure inputs therefore hide the defect and mislead the repair model. Applying a reduction restores focus by keeping only the few tokens that matter, and an LLM can generate the reducer automatically, so the entire process is hands-free. In this example, the reduced prompt improves repair accuracy from an incorrect patch to a fully accepted solution, showing how length control, systematic reduction, and LLM-generated tooling work together to overcome the lost-in-the-middle barrier within APR scenarios.

Algorithm 1 ReduceFix end-to-end workflow

Require: the task description P, reference correct code A, buggy code s_w, hidden test suite $I = \{i_1, \ldots, i_n\}$, failure-inducing input i_0
Ensure: Patched program $\hat{s}$ or *failure*

Phase 1: Reducer Generation
Build one-shot prompt Π using a concrete example
$R \leftarrow$ LLM_Call(Π)
if StaticCheckFail(R) **then**
  return *failure*
end if
Phase 2: Input Reduction
$i^\dagger \leftarrow R$.reduce(i_0) {within 60 seconds}
if Timeout/RE **or** $A(i^\dagger) = s_w(i^\dagger)$ **or** $|i^\dagger| \geq |i_0|$ **then**
  $i^* \leftarrow i_0$ {forward the original failing input}
else
  $i^* \leftarrow i^\dagger$ {use the reduced input}
end if
Phase 3: Patch Generation
$\hat{s} \leftarrow$ LLM_Call($P,\ s_w,\ i^*$)
if $\forall\, i \in I : A(i) = \hat{s}(i)$ **then**
  return $\hat{s}$
end if
return *failure*

3 Approach

3.1 Overview

ReduceFix receives five inputs: the task description P, a correct reference solution A, a buggy submission s_w, the hidden test suite I, and one failure-inducing input i_0. Its goal is to shrink the failure-inducing input i_0 that still distinguishes A from s_w, then guide an LLM to repair the bug.

The pipeline proceeds in three stages, shown in Figure 2. **Reducer Generation** prompts a code LLM once and returns a customized reducer script that is able to automatically reduce the given failure-inducing input for the task. **Input Reduction** executes the generated reducer script under a time limit to shrink the failure-inducing input i_0 into a reduced test input i^*. **Patch Generation** embeds $\langle P, s_w, i^* \rangle$ in a repair prompt, samples candidate patches, and validates each one against the entire test suite I until a correct program $\hat{s}$ is found or the attempt stops.

Algorithm 1 lists the full control logic of ReduceFix.

3.2 Reducer Generation

Instead of directly reducing test inputs using LLMs, ReduceFix leverages one-shot learning to inject the knowledge of the *ddmin* algorithm [48] into the LLM. This enables the model to adapt a well-designed reduction algorithm to various input formats and produce effective, customized reducers.

More specifically, ReduceFix builds one prompt that joins the full task statement P, a single-shot example drawn from task `ABC330D` [8] together with its working reducer, and a few I/O pairs from the current task. The prompt, as shown in Listing 1, is sent to Qwen2.5-Plus [6] with a temperature of 0 (greedy decoding). The model

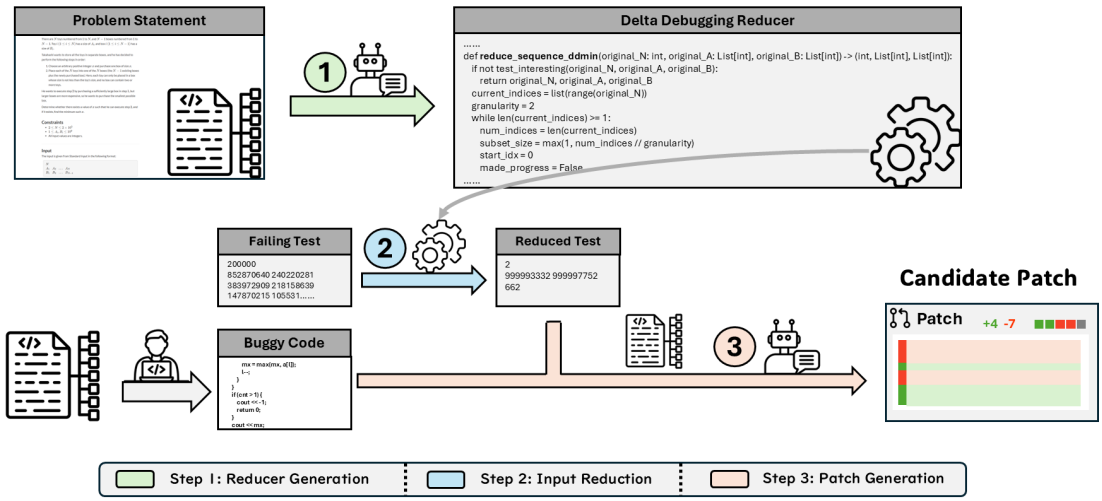


Figure 2: Overview of ReduceFix.

returns a reducer R, which is a customized reducer derived from the *ddmin* algorithm [48].

```
First, here is a complete working example for problem {EXAMPLE_PROBLEM_ID_STR}:
# Example Problem Information ({EXAMPLE_PROBLEM_ID_STR})
## Title: {example_problem_title}
## Problem Description (Markdown)
{example_problem_description_md}
## Example `reducer.py` for {EXAMPLE_PROBLEM_ID_STR}
```python
# START --- Example {EXAMPLE_PROBLEM_ID_STR}/reducer.py --- START
{example_reducer_code}
# END --- Example {EXAMPLE_PROBLEM_ID_STR}/reducer.py --- END
```

Now, using the example above as a reference for structure and helper functions (like `run_program`), please generate the `reducer.py` script for the following target problem:
Target Problem Information ({target_problem_id_input})
Title: {target_problem_title}
Problem Description (Markdown)
{target_problem_description_md}
Generate the complete `reducer.py` code for problem {target_problem_id_input}. Remember to output only the Python code block.
```

Listing 1: One-shot reducer prompt fed to Qwen2.5-Plus

3.3 Input Reduction

In this stage, the LLM-generated reducer will iteratively shrink the given failure-inducing input i_0 while preserving the failure, i.e., the output inconsistency between the accepted reference solution A and the buggy submission s_w. More specifically, the reducer begins by dividing the input into chunks and systematically attempts to produce smaller inputs by removing parts of the original input. For a produced smaller input i, the reducer checks whether it could still cause the inconsistency between the reference correct code A and buggy code s_w, i.e., feeding that input i to both A and s_w and evaluating whether their outputs are different. If it does, it continues the reduction iteration by replacing the original input with a smaller input; if not, it increases the granularity by splitting the original input into more chunks. This process repeats until no further reduction is possible.

Formally, the reducer seeks

$$i^* = \arg\min_{i \preceq i_0} |i| \quad \text{s.t.} \quad A(i) \neq s_w(i), \tag{1}$$

where the notation $i \preceq i_0$ means that i is a subsequence of i_0, or equivalently, i can be obtained from i_0 by deleting elements in i_0 while preserving the relative order of the remaining elements.

The reduction result is a reduced input i^*. The effectiveness of the reducer is measured by the compression rate, defined as the ratio of the size reduction: $\rho = 1 - \frac{|i^*|}{|i_0|}$, where $|i_0|$ and $|i^*|$ denote the sizes of the original and reduced input, respectively.

In our settings, the reducer runs reduction iteration for at most 60 seconds. If the reducer times out, crashes, or cannot shorten the input, the original failure case i_0 is forwarded to the next phase.

3.4 LLM-Guided Repair

In this stage, ReduceFix first concatenates the task description P, the buggy submission s_w and the reduced failure-inducing input i^* in a repair prompt, utilizes LLM to samples candidate patches and validates each one until a correct patched program $\hat{s}$ is found.

Because of LLMs' context length constraints, input prompts have a limited budget. Due to the fact that reduced test input i^* is relatively small, most of them can be inserted into the repair prompt without modification. When i^* (or its associated output) still exceeds a configurable budget L lines, ReduceFix truncates the literal text shown to the LLM: it keeps the first $\lceil L/2 \rceil$ lines and the last $\lfloor L/2 \rfloor$ lines, and never splits a line in the middle. Truncation affects only the prompt; the complete file is preserved for compilation and test execution, so semantic correctness is not compromised.

During candidate patch sampling, ReduceFix utilizes the LLM to generate at most $N = 10$ candidate patches. Each candidate must compile within ten seconds and is executed against the full hidden suite I. A timeout, runtime error, or wrong answer counts as a failed attempt. The repair step succeeds when the first patched program $\hat{s}$ passes the entire test suite I: $\forall\, i \in I, \quad A(i) = \hat{s}(i)$. If no candidate succeeds, the run is reported as a failure.

If truncation occurred in the inferences of patches, the ellipsis token appears inside the fenced block to signal omitted lines. No

other explanatory text is added, keeping the prompt well below typical context limits even on compact LLMs.

4 Experimental Setup

4.1 Models

The selection of evaluated LLMs balances two practical factors: the ability to run an LLM locally and the availability of cloud endpoints with competitive token pricing. For the consumer-GPU category, we select **GLM-4-9B-chat** [4] and **Qwen2.5-Coder-7B-instruct** [6]. Both are fully open-source, fit comfortably on a single 24 GB consumer GPU, and therefore incur no API costs. To represent low-cost hosted offerings, we add **Qwen2.5-Plus** [6] and **DeepSeek-V3** [16]. Qwen2.5-Plus is a closed-weight variant of the Qwen2.5 family (the provider does not disclose an exact parameter count) and is priced at $0.11 per million input tokens and $0.27 per million output tokens. DeepSeek-V3 is larger (671B parameters, 37B active during decoding) yet still affordable at $0.27 per million input tokens and $1.11 per million output tokens. All select LLMs are the standard chat versions rather than the more expensive thinking variants (such as DeepSeek-R1), ensuring a fair and comparable inference budget. This selection enables us to examine how input reduction behaves on both locally deployed LLMs and cost-efficient cloud LLMs.

4.2 Benchmark

A fair study of input reduction for automated program repair (APR) requires two features that existing datasets lack: (1) *long failure-inducing test inputs* and (2) *low risk of training leakage*. Widely used suites such as Defects4J [10], Human-EvalFix [23], and TutorCode [43], Codeflaws [30], and the subset of LeetCode collected by Fan [2] primarily contain only tiny tests, usually a few hundred characters, thus they do not reveal how well an APR pipeline copes when the failing input grows to tens of kilobytes. Meanwhile, most of those benchmarks were released years ago and are drawn from popular open-source projects that large language models have almost certainly seen, which can overstate repair accuracy. To our knowledge, no existing benchmark simultaneously offers long failure-inducing tests and low leakage risk; therefore, we need to build a new benchmark to fill this gap.

To meet the two requirements, the data source must publish every test file, including the largest hidden cases, and it must appear after the training cut-off dates of selected LLMs. LeetCode and Codeforces (including its variant Codeflaws) do not satisfy the first point because they do not share or share only small samples. AtCoder, by contrast, released the full test archives for every Beginner Contest (ABC) up to ABC 377, providing us with large inputs along with an official oracle. Therefore, we select all the satisfied tasks from ABC contests numbered 361 to 377, a span entirely after the knowledge cut-offs of the 4 LLMs we evaluate. From each contest, we retain tasks whose largest official test file is at least 4 KB and whose difficulty level is between C and F, ensuring that the tasks remain solvable for LLMs. To make input diversity explicit, we also categorize the input formats into 6 families and report coverage in Table 1. For cross-language validation, we additionally prepare LFTBench-Py, which mirrors the same 20 tasks and reuses the official tests; each task contributes one wrong-answer Python submission. For each task, we manually collect 10 C++ and 1 Python submissions that failed on a large test before July 1, 2025, resulting in 200 and 20 bugs, respectively. The median failing-input size is over 1 MB, and the largest single file exceeds 8 MB, which is large enough to trigger the "lost-in-the-middle" effect.

Table 1: Input-format categories with counts and tasks within LFTBench.

Category	#Tasks	Tasks
Single sequence	8	361C, 367D, 368C, 369D 370D, 373E, 377C, 377F
Multiple sequences	4	374E, 375C, 376C, 376E
Sequence with dependencies	4	364D, 366C, 372E, 371D
2-D matrix	1	363E
Graph	2	362D, 376D
String	1	365D

We also include a subset of OSS-Fuzz to cover repository-level defects. OSS-Fuzz continuously exercises project-specific fuzzers across many codebases and yields triaged, reproducible crash inputs with sanitizer stacks, which improves external validity and naturally exposes long and irregular inputs [5]. Although some OSS-Fuzz repositories may overlap with LLM pretraining corpora, we hold the LLM and evaluation instances fixed across strategies, so relative differences between prompt types remain valid despite potential data leakage.

4.3 Metrics

A reduction is successful when it completes within 60 seconds, and the resulting input still reproduces the bug. We report (i) the success rate and (ii) the median and average compression rate, as shown in Eq. **??**.

A repair attempt succeeds when at least one candidate patch passes the full test suite (Eq. **??**). For each bug b, we generate n_b candidates once and observe c_b passes. We report

$$\text{pass@}k = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \left(1 - \frac{\binom{n_b - c_b}{k_b}}{\binom{n_b}{k_b}} \right), \quad k_b = \min(k, n_b).$$

This treats the k samples as drawn without replacement and ignores ordering. We report *pass@1*, *pass@5*, and *pass@10*. These three cut-offs align with how developers inspect automated suggestions in practice. Kochhar *et al.* [13] observe that most developers stop using a debugging tool if it does not help within the first five attempts, and Noller *et al.* [25] find that few users review more than ten ranked patches. The same thresholds are widely adopted in recent APR work [3, 7, 19, 45]. Thus *pass@1* reflects a one-shot setting, while *pass@5* and *pass@10* model realistic batch sizes that can be screened offline without taxing developer patience.

4.4 Hyperparameters

Table 2 lists every fixed setting used in our experiments. The hyperparameters fall into two operational blocks: reducer generation (including its subsequent *ddmin* search) and repair inference. All values were chosen with small pilot runs on tasks outside the benchmark and kept unchanged throughout the study.

Table 2: Key hyper-parameters in ReduceFix.

Stage	Parameter	Setting
Reducer generation	LLM backend	Qwen2.5-Plus
	Decoding temperature	0.0 (greedy)
	Wall-clock limit	60s per reduction
Repair inference	# Samples per bug (pass@k)	$k = \{1, 5, 10\}$
	Decoding temperature	0.8
	Compilation timeout	10s
	Execution timeout	5s per test case

To facilitate replication, the full artifact is published at
https://github.com/GLEAM-Lab/ReduceFix

5 Experiments & Results

5.1 Research Questions

- **RQ-1: How reliable are LLM-generated reducers at shrinking failure-inducing inputs?** We assess the ReduceFix reduction phase through two metrics: the *success rate* and the *compression ratio*. Both metrics are evaluated against the unreduced original failure-inducing input, a *ddmin*-only approach, and a purely LLM-based input reduction approach.
- **RQ-2: Does supplying the reduced counterexample improve LLM-based repair?** Using 4 LLMs, we test 3 prompting conditions: Baseline (no failure-inducing input), Origin Test (the full failure-inducing input), and Reduced Test (the reduced input produced by ReduceFix) on LFTBench, and report pass@k for $k \in \{1, 5, 10\}$. To assess cross-language portability, we also run ReduceFix on LFTBench-Py with Qwen2.5-Plus under the same settings.
- **RQ-3: How does prompt composition influence repair accuracy?** We study 3 different prompt variants: Reduced Test, Diff Lines, and Reduced + Origin Test, and compare their average lengths and pass@k. This analysis distinguishes between the benefits of shorter prompts and the benefits of reduced information.
- **RQ-4: Can reduced-input prompting complement existing LLM-based APR pipelines?** We integrate the ReduceFix into the ChatRepair [38] and CREF [43] without modifying its logic. Comparing the augmented version against the original one measures whether input reduction provides a drop-in boost for third-party APR systems.
- **RQ-5: How well does our approach perform on realistic repository-level repair scenarios?** We evaluate ReduceFix end-to-end across OSS-Fuzz projects [5]. For reduction, we compare ReduceFix against the *ddmin*-only and pure-LLM baselines. For repair, we compare pass@k across three prompt settings, the same as RQ-2, to assess whether reduced inputs improve repair accuracy.

5.2 RQ-1: Effectiveness of LLM-generated Reducer

[Objective]: We evaluate whether an LLM can automatically generate a reducer that preserves program failure while reducing the test input. The goal is to determine both the reliability of the generated reducer and the added value of pairing it with the *ddmin* search, as opposed to relying solely on pure LLM test reduction.

[Experimental Design]: We assess the reduction performance on LFTBench, which contains 200 buggy C++ submissions drawn from 20 AtCoder Beginner Contest tasks. For each task, we prompt Qwen2.5-Plus once, using a one-shot example (ABC330D) and the problem statement of the task to produce `reducer.py` (see Section 3.2). The script then runs a *ddmin* loop on the full failure-inducing test input to find a smaller input that still causes the output of the buggy and reference programs to differ. To test whether *ddmin* is necessary, we add a baseline named **pure-LLM**, where the same LLM tries to generate a shorter test input directly. Furthermore, we introduce a baseline named ***ddmin*-only**, which directly applies the *ddmin* algorithm to the failure-inducing test input, using spaces and newlines as delimiters.

A reduction is counted as "Success" when it finishes on time without errors, and the failure is preserved, and returns a strictly smaller input.

Table 3: Reducer success and mean compression by difficulty. Compression is averaged over successful reductions.

Difficulty (n)	Success Rate (%)			Mean/Median Compression Rate (%)		
	ReduceFix	Pure LLM	*ddmin*-only	ReduceFix	Pure LLM	*ddmin*-only
C (60)	100.0	63.3	30.0	84.5/100.0	100.0/100.0	100.0/100.0
D (80)	96.2	36.2	17.5	97.0/100.0	100.0/100.0	100.0/100.0
E&F (60)	88.3	21.7	65.0	83.0/99.9	99.1/99.2	84.2/99.9
Overall (200)	95.0	40.0	35.5	89.1/100.0	99.8/100.0	91.3/100.0

Table 4: Reducer success and mean compression by input format. Compression is averaged over successful reductions.

Input format (n)	Success Rate (%)			Mean/Median Compression Rate (%)		
	ReduceFix	Pure LLM	*ddmin*-only	ReduceFix	Pure LLM	*ddmin*-only
Single sequence (80)	97.5	36.2	52.5	85.9/100.0	99.9/100.0	100.0/100.0
Multiple sequences (40)	100.0	50.0	22.5	76.8/100.0	99.5/100.0	31.7/0.3
Sequence with dependencies (40)	85.0	22.5	35.0	99.0/100.0	100.0/100.0	100.0/100.0
2-D matrix (10)	90.0	0.0	60.0	99.8/99.7	N/A	99.8/99.8
Graph (20)	95.0	65.0	0.0	100.0/100.0	100.0/100.0	N/A
String (10)	100.0	90.0	0.0	100.0/100.0	100.0/100.0	N/A
Overall (200)	95.0	40.0	35.5	89.1/100.0	99.8/100.0	91.3/100.0

[Experimental Results]: Using the prompt template in Listing 1, Qwen2.5-Plus produced a syntactically valid `reducer.py` for all 200 buggy codes. Hence, all subsequent failures are due to the search phase rather than code-generation errors. Tables 3 and 4 show that the LLM-generated reducers succeed on 95.0% of the 200 buggy codes. All submissions of C-difficulty tasks are successfully reduced, while D-difficulty tasks and E & F-difficulty tasks reach 96.2% and 88.3% success, respectively. 10 of 200 forwarded the unreduced case to the repair phase, and in all 10 cases, the *ddmin* loop did not yield a shorter candidate before the time limit. The figure in README of our repository lists all 20 synthesized reducers, highlighting their structural decompositions and preserved semantic invariants.

Statistics in Tables 3 and 4 report the *compression ratio*, defined as Eq. **??**, which means the percentage of bytes eliminated by the reducer. Under this definition, larger numbers indicate stronger

compression: a value of 100% indicates that the input was reduced to almost nothing, while 0% means no reduction at all. The median removal rate remains near 100% across all difficulty levels, indicating that at least half of the test inputs that induce failures are nearly fully stripped yet still reproduce the bug. Averaged by difficulty, the reducer removes 83.0% of bytes on E & F-difficulty tasks, leaving 17.0% of the original data. Tasks in the C-difficulty and D-difficulty groups achieve even larger average removals of 84.5% and 97.0%, respectively.

Tables 3 and 4 show that directly conducting *ddmin* on the input achieves only a 35.5% success rate. Since input formats are often diverse, directly applying *ddmin* often results in a large number of invalid format attempts and thus exhibits a low reduction success rate. Tables 3 and 4 also show what happens when the LLM is leveraged to generate a shorter test input in a one-shot setting, without the pipeline of ReduceFix. This pure LLM baseline succeeds on only 40.0% of the bugs overall and 36.2% of the D-difficulty tasks, and by input type ReduceFix remains robust on graph and string cases where *ddmin*-only fails.

The pure LLM approach, where the LLM tries to generate a shorter input in one pass, consumes about 5.7 million input tokens in total. The ReduceFix requires only 0.094 million input tokens for the same dataset. At current API prices, this comparison translates to $0.632 versus $0.017, 98% saving. Two factors drive this saving. First, the reducer script is generated on a per-task basis and then reused for every buggy submission of this task. Second, prompts in ReduceFix do not include the original failure-inducing input, which is often very large in LFTBench; they contain only the buggy code and the compact candidate input produced by *ddmin*. The pure LLM baseline must embed the entire long test case in the prompt, which greatly inflates the token cost.

Table 5: Pass@K (%) across 4 LLMs.

Difficulty	Baseline (No Test)			Origin Test			Reduced Test		
	@1	@5	@10	@1	@5	@10	@1	@5	@10
Qwen2.5-Coder-7B-instruct									
C	5.5	17.1	23.3	3.7	13.2	20.0	**5.5**	16.7	**25.0**
D	6.4	17.4	25.0	6.1	17.4	23.8	**9.9**	**26.0**	**36.2**
E&F	1.3	5.9	10.0	1.8	7.1	**11.7**	**2.3**	**8.5**	**11.7**
Overall	4.6	13.9	20.0	4.1	13.1	19.0	**6.3**	**17.9**	**25.5**
GLM-4-9B-chat									
C	2.8	5.8	8.3	1.3	3.8	5.0	2.2	4.1	5.0
D	3.5	10.6	13.8	2.0	7.3	11.2	**5.8**	**14.5**	**20.0**
E&F	0.7	1.6	1.7	0.5	1.5	1.7	**1.2**	**1.7**	**1.7**
Overall	2.5	6.5	8.5	1.3	4.5	6.5	**3.3**	**7.5**	**10.0**
Qwen2.5-Plus									
C	37.3	60.3	68.3	31.8	53.5	63.3	33.7	53.5	65.0
D	40.1	55.6	60.0	39.9	**58.7**	61.3	**43.6**	58.7	**62.5**
E&F	22.5	39.7	48.3	24.0	45.7	51.7	**25.0**	**46.5**	**55.0**
Overall	34.0	52.2	59.0	32.7	53.3	59.0	**35.1**	**53.5**	**61.0**
DeepSeek-V3									
C	56.5	76.3	80.0	**56.7**	75.5	78.3	56.3	**77.6**	**81.7**
D	44.5	59.1	62.5	**48.9**	59.2	60.0	48.1	**61.0**	**63.7**
E&F	**35.0**	**53.2**	**58.3**	32.0	50.0	51.7	32.5	51.4	56.7
Overall	45.2	62.5	66.5	**46.1**	61.3	63.0	45.9	**63.1**	**67.0**

[Failed Case Study]: For submission 62869553 of task ABC372E, the student program maintains, for every disjoint-set root, an array that stores the twenty largest vertex identifiers encountered so far. A `Type 1` query merges two components by copying only the ten largest numbers from the other component, then sorts the twenty numbers in descending order. A `Type 2` query asks for the k-th largest vertex inside the root that contains v. When a large component is merged into a smaller one, some very large identifiers disappear; later requests with $k \geq 9$ therefore return incorrect answers.

The generated reducer first applied *ddmin* to discard queries that were not required to trigger the fault, and then renumbered every vertex in the remaining queries to consecutive values $1, 2, \ldots, |V|$. Because the defect depends on the absolute magnitude of vertex identifiers, this renumbering masked the failure, the interestingness predicate became false, and *ddmin* stopped without shrinking the input.

We can resolve the issue by adding one simple guard. After the reducer renumbers the remaining vertices, it immediately reruns the interestingness test. If the failure no longer reproduces, the script rolls back to the original identifiers for that iteration instead of accepting the renumbered version. This small check, implemented in a few lines of Python, prevents the defect from being masked and allows the reduction process to continue correctly without any other modifications.

Table 6: ReduceFix with Qwen2.5-Plus on LFTBench-Py by difficulty.

Difficulty	Baseline (No Test)			Origin Test			Reduced Test		
	@1	@5	@10	@1	@5	@10	@1	@5	@10
C	34.8	62.2	66.7	**38.9**	62.2	66.7	38.3	**64.5**	66.7
D	53.9	69.1	75.0	52.5	62.2	62.5	**61.3**	**84.4**	**87.5**
E&F	**35.8**	**66.7**	**66.7**	18.3	40.3	50.0	35.2	61.5	**66.7**
Overall	42.7	66.3	70.0	38.2	55.6	60.0	**46.5**	**71.5**	**75.0**

[RQ-1] ***[Findings]:*** *(1) ReduceFix successfully reduced on 95% of the 200 bugs, significantly higher than Pure LLM and* ddmin*-only baselines. (2) Reductions delete 80%–97% of inputs on average, leaving a minimal yet still failing input.* ***[Insights]:*** *(1) These results validate the core design of ReduceFix: leveraging LLM to generate a reducer and then driving a systematic ddmin search is both essential and efficient. (2) The success of this "LLM + classic search" pattern suggests that similar hybrids could improve other code-analysis tasks.*

5.3 RQ2: Effectiveness of ReduceFix

[Objective]: We determine whether steering the repair model with the counter-example reduced by ReduceFix improves the accuracy of generating a correct patch compared with (i) no failure-inducing input and (ii) the full failure-inducing input.

[Experimental Design]: We evaluate whether Reduced Test outperforms both Baseline and Origin Test across pass@k for multiple LLMs on LFTBench (C++), and whether this advantage holds on LFTBench-Py (Python) under the same settings. For every one of the 4 selected LLMs (Qwen2.5-Coder-7B-instruct, GLM-4-9B-chat, Qwen2.5-Plus, DeepSeek-V3), we test three prompting modes: Baseline (no test), Origin Test(full failure-inducing test), and Reduced Test. All other hyperparameters are fixed (temperature 0.8,

$k \in \{1, 5, 10\}$, 10 candidate patches per bug), as shown in Table 2. Results are grouped by task difficulty. We also evaluate an end-to-end ddmin-only baseline that uses the reduced input when ddmin succeeds and otherwise falls back to the original failing test, with all other settings unchanged. In addition, for cross-language validation, we evaluate Qwen2.5-Plus on LFTBench-Py under the same protocol; the reducer succeeds on 18 of 20 Python cases (90%).

[Experimental Results]: Across all 4 evaluated LLMs, providing the reduced failure-inducing input consistently raises repair accuracy. Table 5 shows that the overall pass@10 climbs from 20% to 25.5% for Qwen2.5-Coder-7B-instruct, from 8.5% to 10.0% for GLM-4-9B-chat, from 59.0% to 61.0% for Qwen2.5-Plus, and from 66.5% to 67.0% for DeepSeek-V3 when the Reduced Test prompt replaces the Baseline prompt without any other change.

Gains are strongest on D-difficulty tasks. On D-difficulty tasks, the Reduced Test prompt lifts Qwen2.5's pass@10 by 44.8% and pushes GLM-4-9B-chat up by 44.9%. Tasks with difficulties E&F see Qwen2.5-Plus rise from 51.7% to 55.0%, and DeepSeek-V3 recover part of the loss incurred by the origin test input, moving from 51.7% to 56.7%.

In contrast, prompting the unreduced test case often hampers performance. GLM-4-9B-chat's overall pass@10 falls from 8.5% with no test case to 6.5% when the complete input is added, while DeepSeek-V3 drops from 66.5% to 63.0% under the same switch. These observations confirm that a focused counterexample, as provided by ReduceFix, supplies the LLM with sufficient evidence to pinpoint the defect.

To test whether the improvement is due to randomness, we leverage MWW tests [20, 35] to further confirm the improvement based on 4 LLMs, returning a two-sided p-value with < 0.05; therefore, the null hypothesis of equal success probabilities between ReduceFix and the "Origin Test" is rejected, establishing that the gain is statistically significant. On LFTBench with Qwen2.5-Coder-7B-instruct, leveraging reduced test produced by ddmin-only attains 5.7% pass@1, 16.1% pass@5, and 22.0% pass@10, compared to 6.3%, 17.9%, and 25.5% for ReduceFix; the gap at pass@10 is also statistically significant under a two-sided MWW test ($p < 0.05$).

On LFTBench-Py with Qwen2.5-Plus, Reduced Test lifts overall pass@10 to 75.0% versus 70.0% for Baseline and 60.0% for Origin, with the strongest gains on Medium tasks (Table 6).

[RQ-2] ***[Findings]:*** *(1) Supplying the full failing test often hurts repair accuracy; in multiple LLMs and difficulty levels, it performs worse than the no-test Baseline. (2) ReduceFix is consistently better than Origin Test across all LLMs and difficulty groups, and it also outperforms the Baseline in every overall comparison. (3) When evaluating on LFTBench-Py, the improvement of ReduceFix is significant, indicating its cross-language robustness.* ***[Insights]:*** *Trimming a long test while preserving its failure signal boosts APR performance, suggesting that controlled input compression may benefit other long-context tasks that must balance prompt length with information retention.*

5.4 RQ-3: Influence of Prompt Composition on Repair Performance

[Objective]: We investigate the distinct influence of two factors within ReduceFix: (i) length reduction (fewer tokens to keep the bug-relevant text within the model's attention span) and (ii) information selection (retaining the minimal concrete evidence that still exposes the defect).

[Experimental Design]: All settings that are unrelated to the prompt remain identical to RQ-2: the benchmark, the Qwen 2.5 Coder-7B-instruct model, the decoding temperature of 0.8, and the sampling times of ten candidate patches per bug. Five prompt variants cover every combination of the two dimensions under study, and an additional control is included with no test information. The *Baseline* prompt contains only the problem statement and the buggy code. *Diff Lines* stays the same length but appends up to ten mismatched output lines, providing sparse evidence without increasing size. *Reduced Test* includes the full input–output pair of the minimized counterexample; it represents the joint action of length control and full information and is the default in ReduceFix. *Origin Test* swaps the reduced input for the unreduced failure-inducing case, inflating the prompt to about 30.6 KB while holding informational content constant, thereby isolating the cost of extra length. *Reduced + Origin* concatenates the reduced and full tests, introducing redundant information as well as maximum length.

For every bug, the LLM is called exactly once with each variant. After generation, we compile and run the candidate patches against the full hidden test suite and record pass@k for $k \in \{1, 5, 10\}$. Mean prompt length is reported alongside pass@k for each difficulty group (C, D, E&F).

Table 7: Prompt length statistics for prompting strategies.

Strategy	Mean Lines	Mean KBs
Baseline	130.1	3.2
Origin Test	2381.4	30.5
Diff Lines	132.6	3.2
Reduced Test	514.0	6.6
Reduced and Origin Test	2637.7	36.4

[Experimental Results]: Table 7 confirms that the five prompt variants span more than an order of magnitude in length, from roughly 3 KB for *Baseline* and *Diff Lines* to 36 KB for *Reduced + Origin*. Repair accuracy tracks these length and content differences in a highly systematic way (Table 8). Across the full 200-bug benchmark, *Reduced Test* delivers the best outcomes, reaching 25.5% pass@10. This score is 5.5 percentage points higher than the *Baseline* that omits test evidence and 5.5 points higher than the *Origin Test* that embeds the unreduced input.

Performance advantages increase as difficulty rises. On the D-difficulty tasks, *Reduced Test* attains 36.2% pass@10, outperforming both *Diff Lines* (25.0%) and *Origin Test* (23.8%). On the hardest E&F-difficulty, it records 11.7% pass@10, almost doubling the 6.7% achieved by *Diff Lines* and more than tripling the 3.3% scored by *Reduced + Origin*.

Two clear patterns emerge. First, inserting the entire failure-inducing input without reduction inflates the prompt by roughly 30 KB and consistently depresses success, confirming that excessive length dilutes attention. Second, supplying only a terse diff

helps little once problems become non-trivial because the LLM lacks a concrete input–output correspondence. The conjunction of compact length and complete counter-example information is therefore essential; either ingredient alone is insufficient. These findings demonstrate that ReduceFix's prompt strategy offers the best trade-off between information richness and context length, a conclusion likely to generalize to other long-context code-related tasks.

[RQ-3] ***[Findings]:*** *(1) Both Diff Lines and Reduced + Origin fall short of the Reduced Test: overall pass@10 drops from 25.5% to 20.0% for Diff Lines, and to 19.0% for Reduced + Origin. (2) Diff Lines is consistently superior to Reduced + Origin; the gap is most pronounced on C-difficulty tasks, where pass@10 reaches 26.7% versus 23.3%.* ***[Insights]:*** *Simply shortening the prompt can yield noticeable gains, yet the choice of* how *to compress matters. Experimental results underscore that ReduceFix 's reducer design, which is not length reduction alone, is crucial for the observed accuracy improvements.*

Table 8: Pass@k (%) by difficulty under three prompt strategies.

Difficulty	Reduced Test			Diff Lines			Reduced + Origin		
	@1	@5	@10	@1	@5	@10	@1	@5	@10
C	5.5	16.7	25.0	**6.7**	**20.7**	**26.7**	4.7	16.7	23.3
D	**9.9**	**26.0**	**36.2**	6.6	17.7	25.0	6.2	19.1	27.5
E&F	**2.3**	**8.5**	**11.7**	1.5	5.1	6.7	0.5	2.1	3.3
Overall	**6.3**	**17.9**	**25.5**	5.1	14.8	20.0	4.0	13.3	19.0

5.5 RQ-4: Extending ReduceFix to Other APR Pipelines

[Objective]: We validate the extensibility of ReduceFix by adding it as a plug-in to the state-of-the-art APR system ChatRepair and CREF, and observing whether the straightforward replacement of the full failure-inducing input with the reduced counter-example leads to a higher patch accuracy.

[Experimental Design]: We select ChatRepair [38] and CREF [43] for their open harnesses and drop-in reducer interfaces, which let us replace only the failing test while keeping all other logic unchanged. We fix the sampling budget (k) across settings to isolate the effect of reduced vs. unreduced tests. We keep $k = 10$ across settings to equalize token budgets; increasing k raises absolute pass@k but does not affect our budget-controlled comparisons.

ChatRepair [38] is a conversational repair framework that alternates between a user proxy and an LLM. The first user turn delivers the task description, the buggy C++ program, and a single failing test case. The assistant then proposes a patch, which the testing harness compiles and executes; its verdict (pass or fail) becomes the next user message. A run stops when a patch passes all tests or when the retry budget is exhausted. The original implementation exposes two key hyperparameters: `MAX_RETRY`, which limits the number of feedback rounds, and `length`, which decides how many previous turns are retained in the next prompt.

We evaluate ChatRepair and CREF on the built LFTBench benchmark under two prompting modes:

(a) **Origin Test**: Prompting with the full original failure-inducing input test.
(b) **Reduced Test**: Prompting with the reduced test input produced by the generated reducer.

To isolate the effect of the initial prompt, we fix `MAX_RETRY`= 1 (one feedback round) and set the conversation window size to `length`= 2 turns. The LLM samples up to ten candidate patches per bug; we record *pass@k* for $k \in \{1, 5, 10\}$ and results are grouped by difficulty.

CREF [43] is a conversational repair framework for programming tutors that iteratively refines a patch using tutor guidance and test feedback across turns. Since CREF expects a tutor-provided hint in its first turn but LFTBench does not provide such hints, we omit that turn and run a two-turn variant: we first supply the official AtCoder editorial as the high-level solution description, then provide the failure-inducing test case returned by the harness after validating the first-turn patch as the second-turn feedback.

Table 9: Repair success of ChatRepair with full versus reduced failure-inducing inputs on LFTBench.

Difficulty	ChatRepair			ChatRepair + ReduceFix		
	pass@1	pass@5	pass@10	pass@1	pass@5	pass@10
C	12.2	30.9	41.7	**14.5**	**36.6**	**45.0**
D	12.1	28.2	37.5	**16.2**	**35.6**	**46.2**
E&F	2.3	6.8	10.0	**4.0**	**12.8**	**16.7**
Overall	9.2	22.6	30.5	**12.1**	**29.0**	**37.0**

Table 10: Repair success of CREF with full versus reduced failure-inducing inputs on LFTBench.

Difficulty	CREF			CREF + ReduceFix		
	pass@1	pass@5	pass@10	pass@1	pass@5	pass@10
C	15.8	37.3	45.0	**17.3**	**39.5**	**46.7**
D	18.8	39.4	47.5	**19.4**	**39.7**	**47.5**
E&F	5.5	15.6	21.7	**5.7**	**16.4**	**23.3**
Overall	13.9	31.6	39.0	**14.7**	**32.7**	**40.0**

[Experimental Results]: Table 9 indicates that replacing the original failure-inducing input with the ReduceFix counter-example improves ChatRepair across every difficulty level. The overall pass@10 rises from 30.5% to 37.0%, an absolute gain of 6.5 percentage points and a relative gain of 21.3%. Looking by difficulty, C-difficulty tasks improve from 41.7% to 45.0% (7.9% relative improvement), D-difficulty tasks from 37.5% to 46.2% (23.2% relative improvement), and the hardest E&F tasks from 10.0% to 16.7% (67.0% relative improvement). Similar lifts appear at pass@5 and pass@1, confirming that the benefit is robust across different sampling k. Likewise, CREF benefits from ReduceFix: overall pass@10 rises from 39.0% to 40.0%, with pass@5 from 31.6% to 32.7% and pass@1 from 13.9% to 14.7% on LFTBench (Table 10). These results demonstrate that

reducing the supplied failure-inducing test yields a measurable improvement without requiring any other adjustments to existing APR systems.

[RQ-4] ***[Findings]:*** *Integrating ReduceFix with ChatRepair increases the overall pass@10 by 21.3% relative and with CREF by 2.6% to the original configuration.* ***[Insights]:*** *ReduceFix can be adopted as a lightweight add-on for any other APR system that leverages test cases, providing an immediate boost in repair effectiveness while requiring no changes to the existing APR pipeline.*

Table 11: Test case reduction: success rate and compression ratio comparison (Qwen2.5-Plus). SR: Successful Rate, CR: Compress Rate.

Project (n)	*ddmin*-only		ReduceFix		Pure LLM	
	SR	CR	SR	CR	SR	CR
FFmpeg (1)	0.0	N/A	100.0	35.4	0.0	N/A
ImageMagick (1)	100.0	4.6	100.0	5.0	0.0	N/A
MuPDF (5)	100.0	88.1	100.0	87.4	0.0	N/A
PHP-src (1)	100.0	85.6	100.0	97.7	0.0	N/A
Poppler (4)	50.0	22.8	75.0	25.4	0.0	N/A
Overall (12)	75.0	51.8	91.7	56.4	0.0	N/A

Table 12: Pass@*K* of ReduceFix (Qwen2.5-Plus) on OSS-Fuzz.

Project (n)	Baseline			Origin Test			Reduced Test		
	@1	@5	@10	@1	@5	@10	@1	@5	@10
FFmpeg (1)	0.0	0.0	0.0	0.0	0.0	0.0	10.0	50.0	100.0
ImageMagick (1)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
MuPDF (5)	10.0	19.9	20.0	4.0	15.6	20.0	4.0	20.0	40.0
PHP-src (1)	60.0	100.0	100.0	0.0	0.0	0.0	100.0	100.0	100.0
Poppler (4)	25.0	25.0	25.0	22.5	25.0	25.0	25.0	25.0	25.0
Overall (12)	17.5	25.0	25.0	9.2	14.8	16.7	19.2	29.2	41.7

5.6 RQ-5: Experiments on OSS-Fuzz

[Objective]: We aim to evaluate the effectiveness of ReduceFix in realistic software-level repair scenarios beyond C++ programming-exercise benchmarks. We validate ReduceFix on real OSS-Fuzz crash instances, assessing both the robustness of the proposed input-reduction mechanism in large, noisy industrial environments and its cross-language applicability across C/C++ and Bash codes.
[Experimental Design]: We evaluate a guided repair approach for automated fixing of instances in OSS-Fuzz projects [5]. We reuse the data and scripts prepared by ARVO [21] to extract ground-truth patches, failure-inducing inputs, codebases, and issue descriptions, and to evaluate generated patches using pre-built Docker images. The repair pipeline operates iteratively according to the settings in Table 2, except that the reduction phase uses a 600-second time limit to account for Docker test execution overhead. Each LLM invocation receives structured contextual information, including crash diagnostics, complete source files extracted from vulnerable repositories, and failure-inducing input in hexadecimal format. The prompt enforces strict output formatting, including SEARCH/REPLACE blocks with file paths and line numbers for automated patch generation, consistent with prior repository-level repair frameworks [36, 44]. Each generated patch is immediately validated in the original Docker environment provided by ARVO. For *ddmin*-only, we use whitespace/newline tokens for line-oriented textual inputs as in Section 5.2; for binary inputs, we adopt fixed 256-byte blocks with a byte-level fallback. We report the best 1-minimal candidate found within the time budget.

To ensure comparability and avoid post-hoc cherry picking, we first filter instances from ARVO whose ground-truth patch modifies exactly one file and whose failure-inducing input exceeds 1 KB. From the pool that satisfies these constraints across five projects (FFmpeg, ImageMagick, Poppler, php-src, MuPDF), we then select the 12 instances with the **smallest ground-truth patch size** measured by unified diff lines. We compare the effectiveness of reduction across three strategies (ReduceFix, *ddmin*-only, and LLM-generated reduction) by reporting the reduction success rate and compression ratio. Finally, we evaluate the repair accuracy of three strategies (ReduceFix, Baseline without test, and Origin Test) using pass@1, pass@5, and pass@10.
[Experimental Results]: As shown in Table 11 and Table 12, on 12 OSS-Fuzz crashes spanning FFmpeg, Poppler, PHP-src, and MuPDF, ReduceFix preserved the crash during reduction in 91.7% of cases versus 75.0% for *ddmin*-only, a +22.3% relative gain and an absolute +16.7 points. Mean compression rose from 51.8% to 56.4%, a +8.9% relative gain. Meanwhile, *ddmin*-only failed outright on FFmpeg, while ReduceFix both preserved the crash and reduced the input by 35.4%. On PHP-src, compression improved from 85.6% to 97.7% (+14.1% relative) with crash preservation at 100% for both. On Poppler, preservation increased from 50.0% to 75.0% (+50.0% relative) and compression from 22.8% to 25.4% (+11.4% relative). On MuPDF, both preserved the crash, and compression was comparable (-0.8% relative for ReduceFix). The pure-LLM approach fails to reduce any of the failure-inducing test inputs, mainly because these inputs are long (exceeding 1 KB) and structurally complex (e.g., binary file), which LLMs cannot effectively simplify.

Under Docker-grounded validation with Qwen2.5-Plus, reduced tests improved end-to-end repair accuracy. On the 12-sample micro average, pass@10 from 25.0% to 41.7% (+66.8% relative). Using the unreduced Origin Test as the baseline, reduced tests reached 19.2%, 29.2%, and 41.7%, which correspond to +108.7%, +97.3%, and +149.7% relative gains. As an example, in FFmpeg, the ddmin reducer, which deletes fixed-size chunks, often violates the media container length and offset invariants and stops reproducing the failure under the same predicate and harness. In contrast, the LLM-generated reducer applies structure-preserving transformations with adaptive chunking to maintain those invariants before driving ddmin, which preserves the crash and still reduces the input.

[RQ-5] ***[Findings]:*** *ReduceFix is effective on repository-level repair scenarios. On OSS-Fuzz, ReduceFix preserves crashes more reliably than* ddmin*-only while achieving equal or better compression, and those reductions translate into higher pass@k under evaluation. The largest improvements in pass@k occur when* ddmin*-only fails or compresses poorly, including in FFmpeg and PHP-src.* ***[Insights]:*** *By combining semantic-aware LLM reasoning with traditional reduction techniques, ReduceFix reliably preserves failure behavior while producing more concise and interpretable test cases. These reductions substantially improve downstream repair success, confirming that LLM-guided reduction is an essential front-end component of repository-level APR pipelines.*

6 Threats to Validity

Internal validity. LLM outputs are stochastic. Reducer generation uses greedy decoding (temperature = 0), so each prompt yields identical code. Patch generation keeps temperature = 0.8; we sample k patches per bug and report pass@1, pass@5, and pass@10 to reduce sampling variance.

Construct validity. Metric saturation and evaluation bias can distort conclusions. Because ReduceFix's compression ratio often saturates near 100%, we report both mean and median and release the full distribution. To prevent pass@k inflation from partial-test tuning, we validate every patch against the complete official AtCoder test archive. To minimize dataset bias, we include all eligible ABC problems within a fixed date window under our size and difficulty filters.

External validity. To test portability beyond our pipeline, we insert the reducer into ChatRepair and CREF without altering their conversation logic and obtain higher pass@10 on the same benchmark. We also evaluate ReduceFix on LFTBench-Py and on repository-level OSS-Fuzz, where it similarly improves repair performance.

7 Related Work

7.1 LLM-based Program Repair

Program Repair focuses on automatically or semi-automatically fixing software bugs. It aims to reduce the cost and effort of manual debugging by generating patches that correct faulty behavior in code. LLMs have recently advanced automated program repair, significantly surpassing traditional rule-based or search-based techniques [2, 37, 42–44, 46]. ChatRepair is the first work that leverages detailed feedback for each and every patch validated for conversational APR [38]. Following it, many LLM-based repair pipelines embed the failing test case in the prompt to supply bug context [14, 31, 43]. However, when the failure-inducing test case is long, the LLMs' context window is exceeded and attention diffuses, producing the "lost-in-the-middle" effect and lowering patch accuracy [32, 43]. Reducing the failure-inducing input while preserving the failure is therefore crucial for efficient, focused repair, especially when the goal is to fine-tune compact LLMs on highly informative examples. However, no existing work studies test input reduction in the context of APR pipelines. To our knowledge, our work ReduceFix is the first approach that leverages test input reduction techniques into LLM-based APR.

7.2 Test Input Reduction

Reducing test inputs that trigger bugs is crucial for efficient debugging. Delta debugging is the most popular approach for this purpose in software engineering. Zeller and Hildebrandt initially proposed the first delta debugging algorithm, named *ddmin* [47, 48]. Following *ddmin*, HDD [22] and Perses [29] utilize the syntactical structure of the test input to further improve the reduction process. ProbDD [33] introduces a probabilistic model to improve *ddmin*, by estimating the probability of each element being kept in the produced result and prioritizing the reduction of those with high probabilities. GReduce [28] assumes that the given input is generated by a test generator and applies *ddmin* to the generator's execution trace to reduce the test input.

Besides the above delta debugging and its derived algorithms, many domain-specific approaches have been proposed for test input reduction on various domains. For example, CReduce [27], Vulcan [41], and T-Rec [40] reduce source code by iteratively applying pre-defined, well-crafted program transformation rules. LPR [49] and SimpT5 [34] also focus on source code reduction, proposing an LLM-driven technique based on semantic-preserving program transformations. Binary Reduction [11] is proposed to reduce Java bytecode. Those approaches are designed for specialized scenarios in which the test inputs take specific forms, and are thus not applicable to subjects used in our study.

To perform reduction on various types of input formats, such as those in our benchmark tasks, requires manually customizing delta debugging algorithms or designing domain-specific approaches, which is time-consuming and requires domain expertise. In contrast to domain-specific approaches, our work is applicable to a broader range of input types by leveraging LLM's ability to understand specification descriptions. Our approach prompts an LLM to automatically generate the input reducer, enabling task-agnostic input minimization that feeds reduced tests directly back to the repair LLMs.

8 Conclusion

Our study addresses two persistent gaps that exist between test-case reduction and LLM-driven automated program repair. First, we demonstrate that LLMs can generate a task-specific reducer from a single prompt, thereby freeing developers from the need for manual customization to accommodate heterogeneous input formats. Second, we place this reduction step within the LLM-based repair loop so that the trimmed failing case provides a precise, high-signal prompt for patch generation. Experiments on the newly built LFTBench benchmark confirm the value of ReduceFix: inputs shrink by up to 100%, and repair success climbs by up to 53.8%. Ablation study shows that the benefit of ReduceFix stems from the combination of brevity and complete failure evidence rather than length trimming alone. An extension experiment further shows that replacing the test input in ChatRepair with the reduced one increases its pass@10 by 21.3%. These improvements demonstrate that integrating input reduction with LLM-based APR materially advances repair effectiveness. Evaluation on OSS-Fuzz projects further shows that ReduceFix can improve repair accuracy on industrial-scale projects.

Because the automated input reduction is a plug-in component for repair, future work can extend ReduceFix into other APR frameworks and even broadly long-context LLM tasks that profit from concise but information-rich prompts.

Acknowledgments

This work has been partly supported by National Natural Science Foundation (Grant Number 62273292), China; by Central Leading Local Science and Technology Development Project of Hebei Province (Grant Number 246Z0804G), China; by Innovation Capability Improvement Plan Project of Hebei Province (22567626H), China.

References

[1] Zimin Chen, Yue Pan, Siyu Lu, Jiayi Xu, Claire Le Goues, Martin Monperrus, and He Ye. 2025. Prometheus: Unified Knowledge Graphs for Issue Resolution in Multilingual Codebases. *arXiv preprint arXiv:2507.19942* (2025).

[2] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated repair of programs from large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1469–1481.

[3] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. VulRepair: a T5-based automated software vulnerability repair. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 935–947.

[4] Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Dan Zhang, Diego Rojas, Guanyu Feng, Hanlin Zhao, et al. 2024. Chatglm: A family of large language models from glm-130b to glm-4 all tools. *arXiv preprint arXiv:2406.12793* (2024).

[5] Google. [n. d.]. Announcing OSS-Fuzz: Continuous Fuzzing for Open Source Software. https://testing.googleblog.com/2016/12/announcing-oss-fuzz-continuous-fuzzing.html

[6] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).

[7] Faria Huq, Masum Hasan, Md Mahim Anjum Haque, Sazan Mahbub, Anindya Iqbal, and Toufique Ahmed. 2022. Review4Repair: Code review aided automatic program repairing. *Information and Software Technology* 143 (2022), 106765.

[8] AtCoder Inc. 2025. D - Counting Ls. https://atcoder.jp/contests/abc330/tasks/abc330_d Accessed: 2025-07-01.

[9] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 1430–1442. doi:10.1109/ICSE48619.2023.00125 ISSN: 1558-1225.

[10] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440.

[11] Christian Gram Kalhauge and Jens Palsberg. 2019. Binary reduction of dependency graphs. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019*, Marlon Dumas, Dietmar Pfahl, Sven Apel, and Alessandra Russo (Eds.). ACM, 556–566. doi:10.1145/3338906.3338956

[12] Christian Gram Kalhauge and Jens Palsberg. 2021. Logical bytecode reduction. In *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 2021*, Stephen N. Freund and Eran Yahav (Eds.). ACM, 1003–1016. doi:10.1145/3453483.3454091

[13] Pavneet Singh Kochhar, Xin Xia, David Lo, and Shanping Li. 2016. Practitioners' expectations on automated fault localization. In *Proceedings of the 25th international symposium on software testing and analysis*. 165–176.

[14] Jiaolong Kong, Mingfei Cheng, Xiaofei Xie, Shangqing Liu, Xiaoning Du, and Qi Guo. 2024. Contrastrepair: Enhancing conversation-based automated program repair via contrastive test case pairs. *arXiv preprint arXiv:2403.01971* (2024).

[15] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Commun. ACM* 62, 12 (2019), 56–65.

[16] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[17] Kui Liu, Shangwen Wang, Anil Koyuncu, Kisub Kim, Tegawendé F Bissyandé, Dongsun Kim, Peng Wu, Jacques Klein, Xiaoguang Mao, and Yves Le Traon. 2020. On the efficiency of test suite based program repair: A systematic assessment of 16 automated repair systems for java programs. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 615–627.

[18] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172* (2023).

[19] Wenqiang Luo, Jacky Wai Keung, Boyang Yang, Jacques Klein, Tegawende F Bissyande, Haoye Tian, and Bach Le. 2025. Unlocking LLM Repair Capabilities in Low-Resource Programming Languages Through Cross-Language Translation and Multi-Agent Refinement. *arXiv preprint arXiv:2503.22512* (2025).

[20] Henry B Mann and Donald R Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics* (1947), 50–60.

[21] Xiang Mei, Pulkit Singh Singaria, Jordi Del Castillo, Haoran Xi, Tiffany Bao, Ruoyu Wang, Yan Shoshitaishvili, Adam Doupé, Hammond Pearce, Brendan Dolan-Gavitt, et al. 2024. Arvo: Atlas of reproducible vulnerabilities for open source software. *arXiv preprint arXiv:2408.02153* (2024).

[22] Ghassan Misherghi and Zhendong Su. 2006. HDD: hierarchical Delta Debugging. In *28th International Conference on Software Engineering (ICSE 2006), Shanghai, China, May 20-28, 2006*, Leon J. Osterweil, H. Dieter Rombach, and Mary Lou Soffa (Eds.). ACM, 142–151. doi:10.1145/1134285.1134307

[23] Niklas Muennighoff, Qian Liu, Armel Randy Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro Von Werra, and Shayne Longpre. [n. d.]. OctoPack: Instruction Tuning Code Large Language Models. In *The Twelfth International Conference on Learning Representations*.

[24] Aina Niemetz and Armin Biere. 2013. ddSMT: a delta debugger for the SMT-LIB v2 format. In *Proceedings of the 11th International Workshop on Satisfiability Modulo Theories, SMT*. 8–9.

[25] Yannic Noller, Ridwan Shariffdeen, Xiang Gao, and Abhik Roychoudhury. 2022. Trust enhancement issues in program repair. In *Proceedings of the 44th International Conference on Software Engineering*. 2228–2240.

[26] Van-Thuan Pham, Wei Boon Ng, Konstantin Rubinov, and Abhik Roychoudhury. 2015. Hercules: Reproducing crashes in real-world application binaries. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 891–901.

[27] John Regehr, Yang Chen, Pascal Cuoq, Eric Eide, Chucky Ellison, and Xuejun Yang. 2012. Test-case reduction for C compiler bugs. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '12, Beijing, China - June 11 - 16, 2012*, Jan Vitek, Haibo Lin, and Frank Tip (Eds.). ACM, 335–346. doi:10.1145/2254064.2254104

[28] Luyao Ren, Xing Zhang, Ziyue Hua, Yanyan Jiang, Xiao He, Yingfei Xiong, and Tao Xie. 2025. Validity-Preserving Delta Debugging via Generator Trace Reduction. *ACM Trans. Softw. Eng. Methodol.* 34, 3, Article 65 (Feb. 2025), 33 pages. doi:10.1145/3705305

[29] Chengnian Sun, Yuanbo Li, Qirun Zhang, Tianxiao Gu, and Zhendong Su. 2018. Perses: syntax-guided program reduction. In *Proceedings of the 40th International Conference on Software Engineering, ICSE 2018, Gothenburg, Sweden, May 27 - June 03, 2018*, Michel Chaudron, Ivica Crnkovic, Marsha Chechik, and Mark Harman (Eds.). ACM, 361–371. doi:10.1145/3180155.3180236

[30] Shin Hwei Tan, Jooyong Yi, Yulis, Sergey Mechtaev, and Abhik Roychoudhury. 2017. Codeflaws: a programming competition benchmark for evaluating automated program repair tools. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*. 180–182. doi:10.1109/ICSE-C.2017.76

[31] Hao Tang, Keya Hu, Jin Zhou, Si Cheng Zhong, Wei-Long Zheng, Xujie Si, and Kevin Ellis. 2024. Code repair with llms gives an exploration-exploitation tradeoff. *Advances in Neural Information Processing Systems* 37 (2024), 117954–117996.

[32] Haoye Tian, Weiqi Lu, Tsz On Li, Xunzhu Tang, Shing-Chi Cheung, Jacques Klein, and Tegawendé F Bissyandé. 2023. Is ChatGPT the ultimate programming assistant–how far is it? *arXiv preprint arXiv:2304.11938* (2023).

[33] Guancheng Wang, Ruobing Shen, Junjie Chen, Yingfei Xiong, and Lu Zhang. 2021. Probabilistic Delta debugging. In *ESEC/FSE '21: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, August 23-28, 2021*, Diomidis Spinellis, Georgios Gousios, Marsha Chechik, and Massimiliano Di Penta (Eds.). ACM, 881–892. doi:10.1145/3468264.3468625

[34] Haibo Wang, Zezhong Xing, Chengnian Sun, Zheng Wang, and Shin Hwei Tan. 2025. Towards Diverse Program Transformations for Program Simplification. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 312–334.

[35] Frank Wilcoxon. 1992. Individual comparisons by ranking methods. In *Breakthroughs in statistics: Methodology and distribution*. Springer, 196–202.

[36] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* (2024).

[37] Chunqiu Steven Xia and Lingming Zhang. 2022. Less training, more repairing please: revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 959–971.

[38] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 819–831.

[39] Junjielong Xu, Ying Fu, Shin Hwei Tan, and Pinjia He. 2024. Aligning the Objective of LLM-based Program Repair. *arXiv preprint arXiv:2404.08877* (2024).

[40] Zhenyang Xu, Yongqiang Tian, Mengxiao Zhang, Jiarui Zhang, Puzhuo Liu, Yu Jiang, and Chengnian Sun. 2025. T-Rec: Fine-Grained Language-Agnostic Program Reduction Guided by Lexical Syntax. *ACM Trans. Softw. Eng. Methodol.* 34, 2, Article 34 (Jan. 2025), 31 pages. doi:10.1145/3690631

[41] Zhenyang Xu, Yongqiang Tian, Mengxiao Zhang, Gaosen Zhao, Yu Jiang, and Chengnian Sun. 2023. Pushing the Limit of 1-Minimality of Language-Agnostic Program Reduction. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 97 (April 2023), 29 pages. doi:10.1145/3586049

[42] Boyang Yang, Zijian Cai, Fengling Liu, Bach Le, Lingming Zhang, Tegawendé F. Bissyandé, Yang Liu, and Haoye Tian. 2025. A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications. arXiv:2506.23749 [cs.SE] https://arxiv.org/abs/2506.23749

[43] Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F Bissyandé, and Shunfu Jin. 2024. Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd*

ACM SIGSOFT International Symposium on Software Testing and Analysis. 882–894.

[44] Boyang Yang, Haoye Tian, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, and Bach Le. 2025. Enhancing Repository-Level Software Repair via Repository-Aware Knowledge Graphs. *arXiv preprint arXiv:2503.21710* (2025).

[45] Boyang Yang, Haoye Tian, Jiadong Ren, Hongyu Zhang, Jacques Klein, Tegawende Bissyande, Claire Le Goues, and Shunfu Jin. 2025. MORepair: Teaching LLMs to Repair Code via Multi-Objective Fine-Tuning. *ACM Transactions on Software Engineering and Methodology* (2025).

[46] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. 2024. ThinkRepair: Self-Directed Automated Program Repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, Vienna Austria, 1274–1286. doi:10.1145/3650212.3680359

[47] Andreas Zeller. 1999. Yesterday, My Program Worked. Today, It Does Not. Why?. In *Software Engineering - ESEC/FSE'99, 7th European Software Engineering Conference, Held Jointly with the 7th ACM SIGSOFT Symposium on the Foundations of Software Engineering, Toulouse, France, September 1999, Proceedings (Lecture Notes in Computer Science, Vol. 1687)*, Oscar Nierstrasz and Michel Lemoine (Eds.). Springer, 253–267. doi:10.1007/3-540-48166-4_16

[48] Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE Trans. Software Eng.* 28, 2 (2002), 183–200. doi:10.1109/32.988498

[49] Mengxiao Zhang, Yongqiang Tian, Zhenyang Xu, Yiwen Dong, Shin Hwei Tan, and Chengnian Sun. 2024. LPR: Large Language Models-Aided Program Reduction. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 261–273. doi:10.1145/3650212.3652126